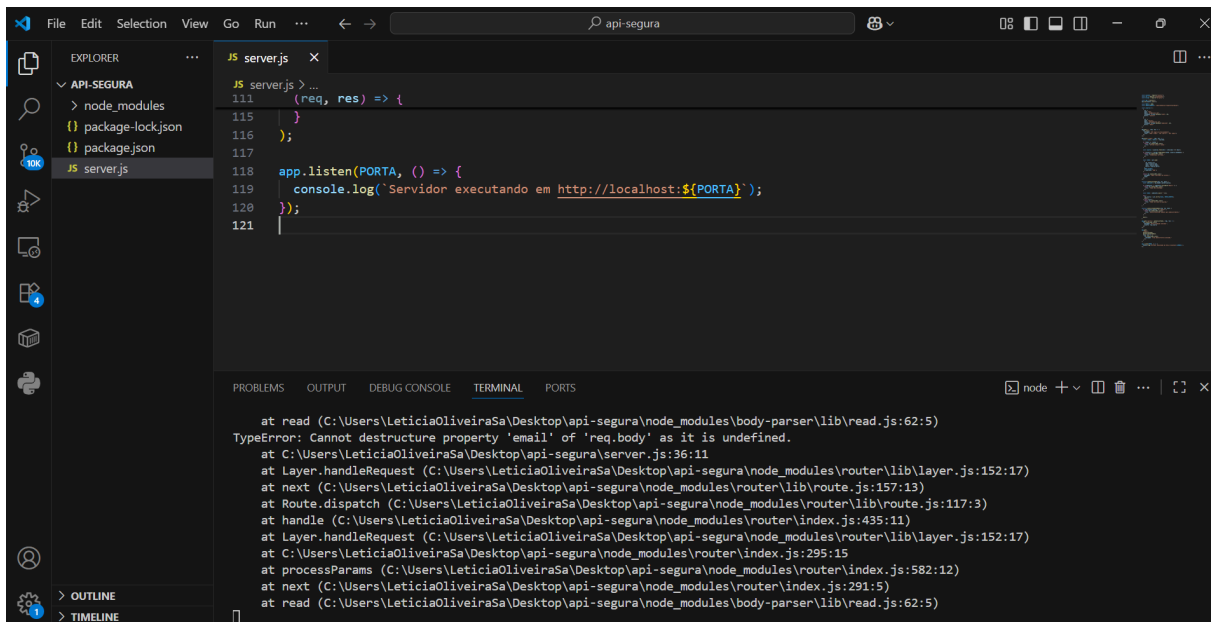


REGISTRO DE BACKEND - SEMANA 15

Print Terminal:

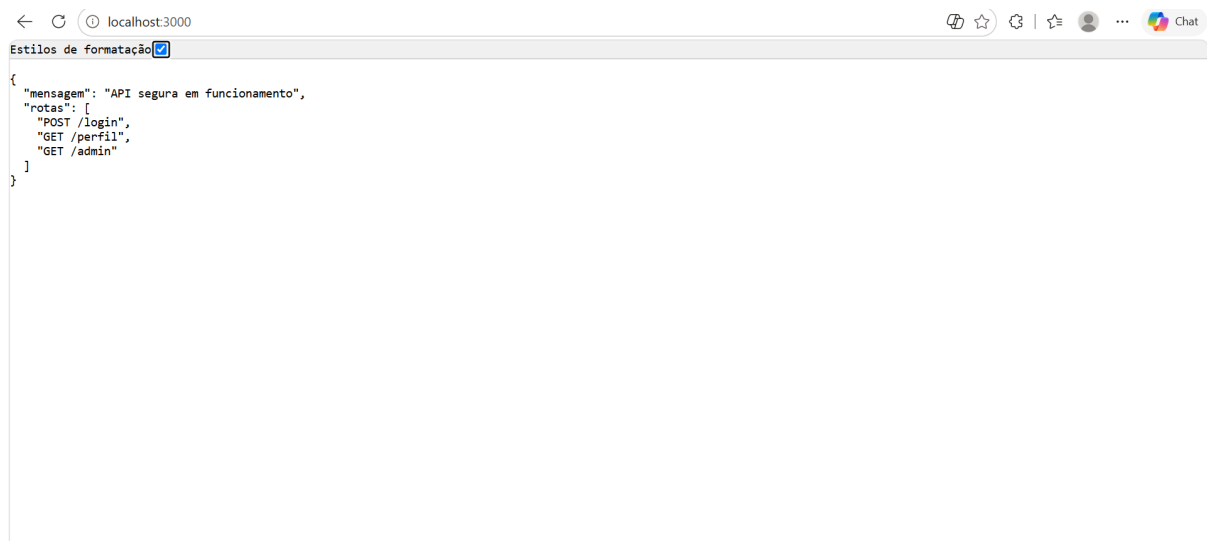


The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'API-SEGURA' with files 'package-lock.json', 'package.json', and 'server.js'. The main editor area displays the content of 'server.js', which includes a Node.js server setup using Express.js. The terminal window at the bottom shows a stack of error messages, starting with 'TypeError: Cannot destructure property 'email' of 'req.body' as it is undefined.' The error trace points to 'body-parser' and 'router' modules.

```
JS server.js
111 (req, res) => {
115 }
116 };
117
118 app.listen(PORTA, () => {
119   console.log(`Servidor executando em http://localhost:${PORTA}`);
120 });
121
```

```
at read (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\body-parser\lib\read.js:62:5)
TypeError: Cannot destructure property 'email' of 'req.body' as it is undefined.
at C:\Users\LeticiaOliveiraSa\Desktop\api-segura\server.js:36:11
at Layer.handleRequest (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\lib\layer.js:152:17)
at next (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\lib\route.js:157:13)
at Route.dispatch (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\lib\route.js:117:3)
at handle (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\index.js:435:11)
at Layer.handleRequest (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\lib\layer.js:152:17)
at C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\index.js:295:15
at processParams (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\index.js:582:12)
at next (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\router\index.js:291:5)
at read (C:\Users\LeticiaOliveiraSa\Desktop\api-segura\node_modules\body-parser\lib\read.js:62:5)
```

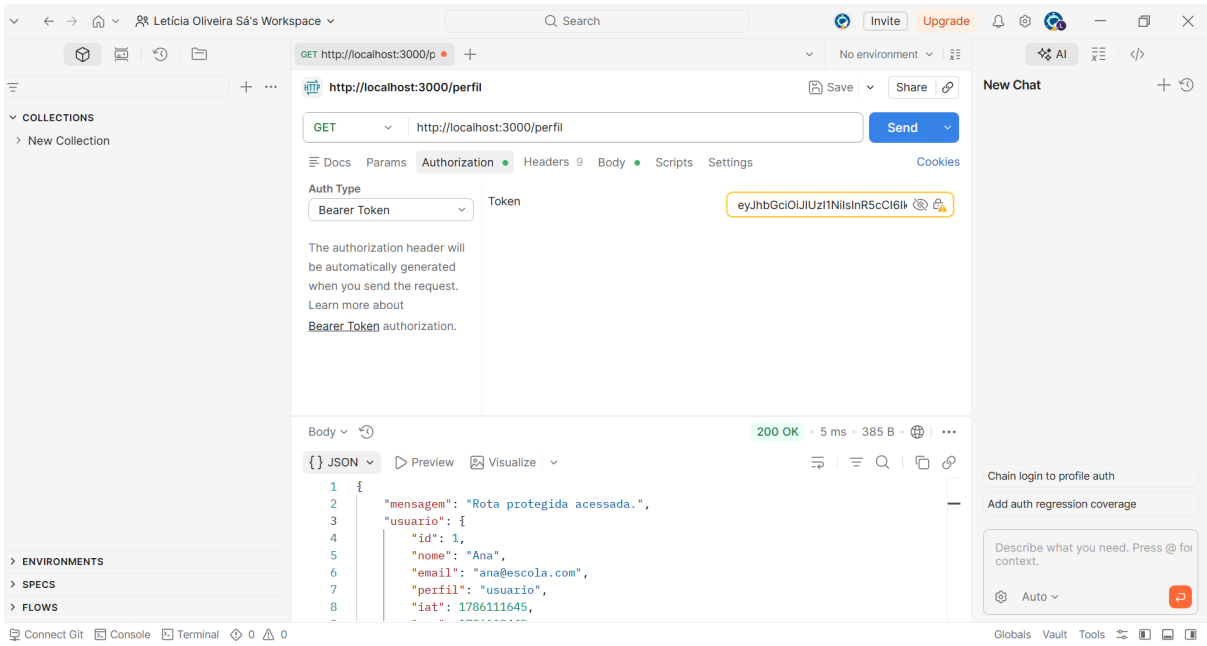
Print localhost (explore):



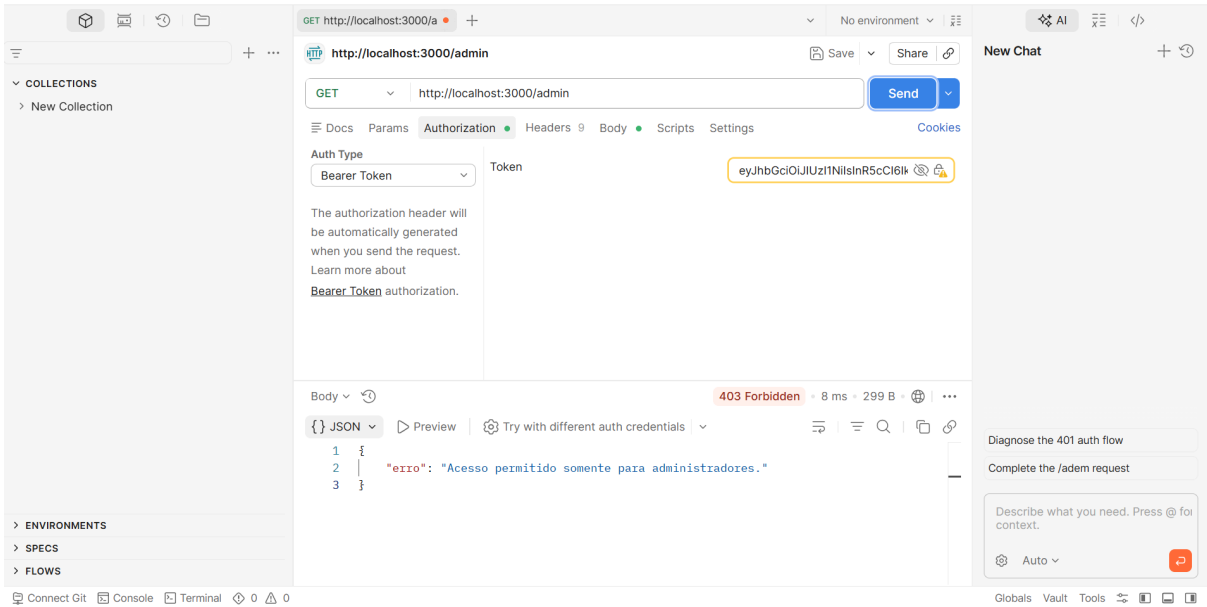
The screenshot shows a web browser window with the address bar set to 'localhost:3000'. The page content displays a JSON object representing the API's response. The JSON includes a 'mensagem' field and an array of 'rotas' (routes).

```
{
  "mensagem": "API segura em funcionamento",
  "rotas": [
    "POST /login",
    "GET /perfil",
    "GET /admin"
  ]
}
```

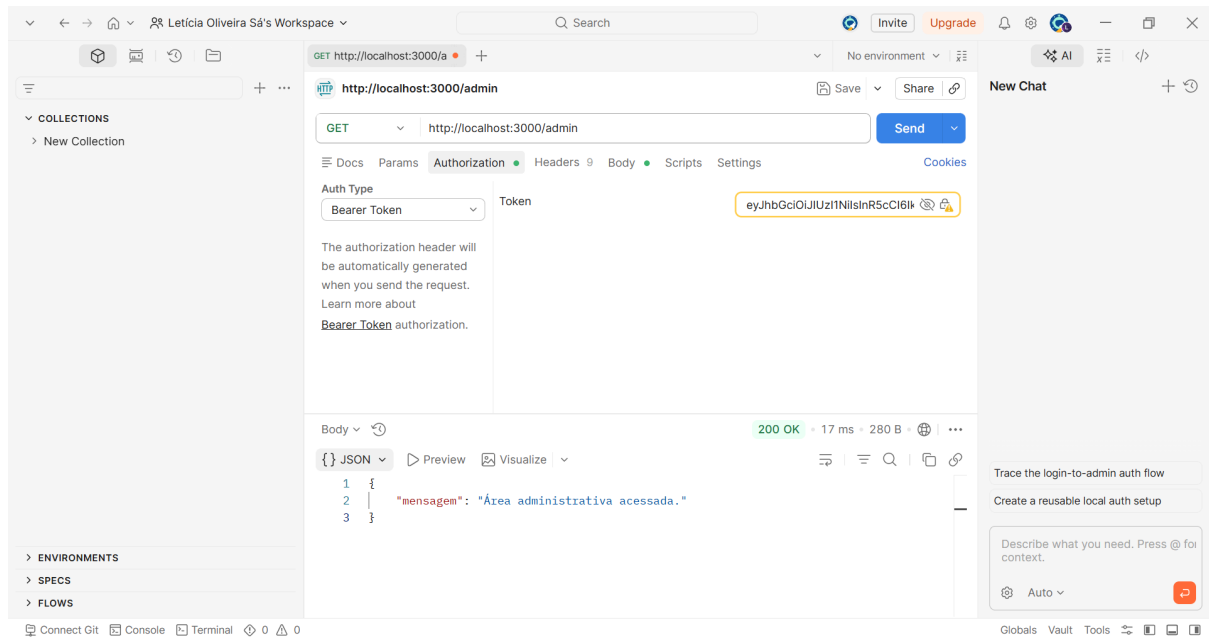
Print Postman (login):



Print Postman (admin (acesso negado)):



Print Postman (admin (acesso liberado)):



Perguntas:

1. Explique, com suas palavras, a diferença entre autenticação e autorização.

Resposta: Autenticação confirma quem é o usuário. Autorização verifica o que ele pode acessar ou fazer.

2. Por que o usuário comum recebe 403 ao acessar a rota /admin?

Resposta: Porque ele está logado, mas não tem permissão de administrador para acessar essa rota.

3. Qual é a função do token JWT nesta atividade?

Resposta: Identificar o usuário depois do login e permitir que o servidor reconheça suas próximas requisições.

4. Por que não devemos colocar senhas dentro do JWT?

Resposta: Porque o conteúdo do JWT pode ser visualizado. Assim, a senha ficaria exposta.

5. O que o bcrypt faz com a senha?

Resposta: O bcrypt transforma a senha em um hash seguro, evitando que ela seja armazenada em texto puro.

6. Qual é a diferença entre dados em trânsito e dados em repouso?

Resposta: Dados em trânsito estão sendo enviados pela rede. Dados em repouso estão armazenados, como em um banco de dados.

7. Por que esta API deve utilizar HTTPS quando for publicada?

Resposta: Para proteger os dados enviados entre o usuário e o servidor contra interceptação.

8. O que os testes 2, 3, 4 e 5 demonstraram sobre a segurança da API?

Resposta: Mostraram que a API bloqueia senhas erradas, exige um token válido e impede acessos sem permissão.